

Estructuras de datos en C++

Programación 2

1er cuatrimestre 2026

1. Clases Simples

Crear una clase que represente un intervalo de números reales. Un intervalo tiene un extremo inferior y un extremo superior, y cada extremo puede ser abierto o cerrado.

El constructor de la clase debe recibir como parámetros los valores de los extremos y si son abiertos y cerrados.

Además implementar las siguientes operaciones:

- Consultar los límites del intervalo y si son abiertos o cerrados
- Consultar si un número está contenido o no en el intervalo
- Comparación de igualdad entre intervalos
- Determinar si dos intervalos se superponen (hay uno o más números que pertenecen a ambos intervalos)
- Determinar si un intervalo está contenido dentro de otro

2. Clases con Arreglos Estáticos

Agenda de contactos

Se desea crear agenda de contactos que implemente las siguientes operaciones:

- Agregar un contacto nuevo
- Editar datos de un contacto existente
- Borrar un contacto
- Acceder a los datos de un contacto (acceso de sólo lectura)
- Consultar la cantidad de contactos

Cada contacto tiene los siguientes datos:

- Nombre
- Apellido
- Número de teléfono
- Dirección de mail

1. Crear una clase `Contacto` que contenga los datos de un contacto y permita editarlos.

Tener en cuenta que todos los datos son opcionales (pueden faltar o estar vacíos), pero el nombre y apellido no pueden estar vacíos a la misma vez.

Los métodos de la clase deben garantizar que los números de teléfono sean válidos (pueden contener solamente dígitos y espacios, y pueden o no empezar con el carácter '+').

2. Definir operadores `<<` y `>>` que permitan escribir un contacto a una salida de texto y leer un contacto de una entrada de texto.

Nota: El formato para escribir los datos lo pueden definir arbitrariamente, pero los datos que se escriben con el operador `<<` se tienen que poder leer con el operador `>>`.

3. Crear una clase `AgendaDeContactos` que implemente las operaciones del principio del enunciado.
4. Definir el operador `<<` que escriba todos los contactos de la agenda a una salida de texto, en orden alfabético.
5. Definir el operador `>>` que lea contactos de una entrada de texto y los agregue a una agenda. En caso de que se lea un contacto con nombre y apellido existente, se debe actualizar el contacto existente; en caso contrario se debe agregar un contacto nuevo.

3. Clases con Memoria Dinámica

1. **Lista Simple Enlazada:** a. Implementar una clase de lista simplemente enlazada con métodos `insertarAlComienzo`, `insertarAlFinal`, `borrarElemento`, `mostrar`. b. Actualizar la lista enlazada para que, al insertar un elemento, lo ubique automáticamente en la posición correcta (insertar ordenado).
2. **Pila Dinámica (Stack):** Implementar una pila con funciones `push`, `pop` y `top` utilizando la lista del punto anterior.
3. **Cola Dinámica (Queue):** Implementar `enqueue` y `dequeue` utilizando la lista del punto anterior.

4. Problemas de Combinación de Estructuras

Elegir un problema e implementar una solución utilizando los tipos de datos conocidos. En cada caso, programar la lógica descripta generar casos de test simulando interacciones con el sistema que abarque la mayor cantidad de casos posibles.

Las ambigüedades del enunciado pueden resolverse con una decisión documentada o charlando con los docentes.

1. **Sistema de Impresión:** Un servidor recibe pedidos de impresión y los encola con la función:
 - `encolar_impresion(nombre_documento)` Además:
 - la función `enviar_impresion(nombre_documento)` retira el próximo pedido de la cola, lo envía al dispositivo y genera un mensaje `[nombre_documento+" impreso OK"]` en una lista de mensajes de Historial.
 - la función `cancelar_impresion(nombre_documento)` quita el documento de la cola (cualquier documento encolado) y genera una entrada en la lista de mensajes de historial con el texto `[nombre_documento+" fue cancelado"]`.
 - La función `reporte_servidor()` debe imprimir los documentos encolados y el historial de mensajes.
2. **Navegador Web:** Implementar el gestor de historial del explorador usando una **Pila** para el botón "Atrás" y otra **Pila** para "Adelante".
 - El sistema arranca con las pilas vacías y con la barra de direcciones vacía.
 - La función `click_link(url)` actualiza el valor e la barra de direcciones y encola la dirección en la pila "Atrás" y resetea la cola "Adelante".
 - La función `click_atras()` encola la dirección de la barra de direcciones en la cola "Adelante" y actualiza la barra de direcciones.
 - La función `click_adelante()` encola la dirección de la barra en la cola "Atrás" y desencola la dirección de la cola "Adelante".
3. **Control de Peaje:** Implementar un mecanismo de **Cola** de vehículos donde los de emergencia deben saltar al principio.
 - Cada vehículo que llega al peaje en circunstancias normales se encola en la barrera.
 - Pueden llegar vehículos con alguna emergencia y no puede esperar. Hay distintos niveles de emergencia.
 - El peaje tiene una función que deja pasar un auto lo que genera que luego de eso se actualice la cola de autos en la barrera.
 - La función que deja pasar el auto, debe permitir adelantarse al auto cuya emergencia sea la de mayor criticidad. Si esto sucede, el auto con *prioridad* pasa por la barrera y el encolado se reacomoda.